University of Colorado Denver

Department of Computer Science & Engineering

CSCI/CSCY 4407: Security & Cryptography

Lab 3 Assignment: Symmetric Encryption: Block Ciphers, IND-CPA, AEAD & Cryptographic Failures

Instructor: Dr. Victor Kebande

Teaching Assistant: Celest Kester

Spring 2026

Assignment Instructions

This is a **group assignment**. Each group is required to complete and submit its own original work. You may discuss general security and cryptography concepts with classmates; however, **you are not permitted to share, reuse, or copy** solutions, calculations, code, scripts, screenshots, written explanations, recovered plaintexts, decrypted outputs, experimental results, or final answers across groups.

For this lab, some tasks require you to **generate your own plaintext files and perform controlled experiments**, while others require you to use **group-specific ciphertext artifacts**.

- **Tasks 1, 2, 3, 5, 6, and 7:** You must generate your own plaintext files, keys, IVs/nonces, and experimental data as instructed in the lab manual.
- **Task 4 (CTR Nonce Reuse Attack):** Each group has been assigned a **unique ciphertext package** generated using different keys and nonces. These files simulate a controlled keystream-reuse scenario.

Because each group receives different cryptographic artifacts, **all recovered outputs and intermediate results will differ across groups**. This is intentional and forms part of the assessment design.

It is your responsibility to ensure that you download and use the correct files assigned to your group. The group-specific files are available on Canvas under:

Modules → Lab 3: Symmetric Encryption: Block Ciphers, IND-CPA, AEAD...

Each group should download only their assigned file:

`Ciphertext_Group_XX.zip`

Using another group's ciphertext package will result in incorrect outputs and may be considered an academic integrity violation. Failure to follow these instructions may lead to grade penalties.

Academic Integrity and Collaboration

- **Allowed:** Discussing lecture material and cryptographic concepts at a high level.
- **Not allowed:** Sharing code, decrypted messages, keys, screenshots, or report content with other groups.
- **Not allowed:** Copying solutions from the internet, generative AI tools, previous students, or public repositories.
- **Submission:** Only **one** group member should submit the final report on Canvas on behalf of the group.

1 Introduction

Modern symmetric cryptography is built upon carefully designed mathematical constructions that transform fixed-size blocks of data into seemingly random outputs under a secret key. At the core of most modern systems is a **block cipher**, such as the Advanced Encryption Standard (AES). A block cipher is a deterministic function defined as:

$$E_k : \{0,1\}^n \to \{0,1\}^n$$

where:

- k is a secret key,
- n is the fixed block size (e.g., $n = 128$ bits for AES),
- E_k is invertible, meaning there exists D_k such that

$$D_k(E_k(P)) = P$$

for every plaintext block $P \in \{0,1\}^n$.

Although block ciphers are deterministic, real-world messages are typically longer than one block. Therefore, block ciphers must be combined with a **mode of operation** to securely encrypt arbitrary-length messages. The security of symmetric encryption depends not only on the strength of the block cipher itself, but on how these modes are constructed and how parameters such as initialization vectors (IVs) and nonces are managed.

1.1 Electronic Codebook (ECB)

In ECB mode, each block is encrypted independently:

$$C_i = E_k(P_i)$$

Because this process is deterministic, identical plaintext blocks produce identical ciphertext blocks:

$$P_i = P_j \Rightarrow C_i = C_j$$

This leakage of structural information violates semantic security and makes ECB unsuitable for secure applications.

1.2 Cipher Block Chaining (CBC)

CBC introduces randomness through an initialization vector (IV):

$$C_1 = E_k(P_1 \oplus IV)$$

$$C_i = E_k(P_i \oplus C_{i-1}) \quad \text{for } i \geq 2$$

Decryption follows:

$$P_i = D_k(C_i) \oplus C_{i-1}$$

If the IV is unpredictable and unique, CBC achieves semantic security under standard assumptions. However, if the IV is reused, the first block reveals relationships:

$$C_1 = E_k(P_1 \oplus IV)$$

Thus, reuse of IV makes encryptions linkable.

1.3 Counter Mode (CTR)

CTR transforms a block cipher into a stream-like construction:

$$C_i = P_i \oplus E_k(\text{Nonce} \parallel \text{Counter}_i)$$

Decryption is identical:

$$P_i = C_i \oplus E_k(\text{Nonce} \parallel \text{Counter}_i)$$

CTR behaves similarly to a stream cipher because encryption consists of XOR with a keystream. However, nonce reuse is catastrophic. If the same nonce is reused for two messages:

$$C_1 = P_1 \oplus K$$

$$C_2 = P_2 \oplus K$$

Then:

$$C_1 \oplus C_2 = P_1 \oplus P_2$$

This recreates the classical **two-time pad** vulnerability.

1.4 Nonce Collisions and the Birthday Bound in CTR

CTR mode critically relies on the assumption that the nonce is **never reused** under the same key. In practice, nonces are often chosen randomly from a finite space of size 2^r, where r is the nonce length in bits. Suppose a system performs q encryptions using randomly selected r-bit nonces. The probability that *at least one nonce collision* occurs is approximated by the birthday bound:

$$\Pr[\text{collision}] \approx 1 - \exp\left(-\frac{q(q-1)}{2 \cdot 2^r}\right)$$

For small probabilities, this simplifies to:

$$\Pr[\text{collision}] \approx \frac{q^2}{2^{r+1}}$$

This implies that collisions become likely when:

$$q \approx 2^{r/2}$$

This phenomenon is known as the **birthday paradox**. Even if r is moderately large, repeated encryption operations can eventually cause nonce reuse with non-negligible probability.

When a nonce collision occurs in CTR mode under the same key, the keystream is reused:

$$E_k(\text{Nonce} \parallel \text{Counter}_i)$$

and the security collapses exactly as in the two-time pad case:

$$C_1 \oplus C_2 = P_1 \oplus P_2$$

Thus, nonce uniqueness is not merely a recommendation; it is a strict requirement for CTR security.

For this reason, modern cryptographic standards require either:

- Strictly increasing counters, or
- Nonce management protocols that guarantee uniqueness.

Failure to enforce nonce uniqueness enables practical attacks even when the underlying block cipher (e.g., AES) remains cryptographically secure.

1.5 Authenticated Encryption (AES-GCM)

Confidentiality alone does not guarantee integrity. Authenticated Encryption with Associated Data (AEAD), such as AES-GCM, provides both:

$$(C, T) = \mathrm{Enc}_k(P, A)$$

where:

- C is ciphertext,
- T is an authentication tag,
- A is optional associated data.

Decryption verifies integrity:

$$\mathrm{Dec}_k(C, T, A) = \begin{cases} P & \text{if tag valid} \\ \perp & \text{otherwise} \end{cases}$$

Any modification of ciphertext or tag results in failure.

2 Security Perspective

The formal goal of modern symmetric encryption is **semantic security**, often defined through the notion of indistinguishability under chosen-plaintext attack (IND-CPA). Informally, an encryption scheme is secure if an adversary cannot distinguish encryptions of two chosen plaintexts with probability significantly better than random guessing. Randomness (via IVs or nonces), correct key management, and authenticated encryption are therefore essential. Failure to follow these principles leads to structural leakage, keystream reuse attacks, padding oracle vulnerabilities, and integrity failures.
In this laboratory, you will move beyond simple encryption and examine how symmetric cryptographic constructions succeed or fail under realistic operational conditions and misuse scenarios. A summary of the tasks to be achieved and the intended objectives is also shown in Table 1

- Construct an ECB distinguisher and demonstrate why deterministic encryption leaks structural information and violates semantic security.
- Analyze CBC mode under both proper and improper IV usage, and evaluate how IV reuse affects linkability and security.

- Implement CTR mode correctly and verify its correctness through experimental validation.
- Exploit nonce reuse in CTR mode to demonstrate the classical "two-time pad" vulnerability and recover plaintext relationships.
- Analyze nonce collision probability using the birthday bound, and experimentally demonstrate how repeated encryptions increase the likelihood of keystream reuse.
- Compare confidentiality-only modes with authenticated encryption (AES-GCM) to demonstrate the necessity of integrity protection.
- Benchmark different encryption modes and evaluate security-performance trade-offs in practical implementations.

Table 1: Overview of Lab Tasks

Task	**Title**	**Primary Learning Objective**
Task 1	ECB Distinguisher	Show why deterministic encryption leaks structure and violates IND-CPA. Detect repeated ciphertext blocks.
Task 2	CBC IV Experiment	Demonstrate how CBC achieves randomized encryption with fresh IVs and how IV reuse creates linkability.
Task 3	CTR Mode Implementation	Implement AES-CTR correctly, verify correctness with hashes, and explain stream-like behavior.
Task 4	CTR Nonce Reuse Attack	Exploit keystream reuse and demonstrate the "two-time pad" failure: $C_1 \oplus C_2 = P_1 \oplus P_2$.
Task 5	Birthday Attack on CTR (Nonce Collision)	Quantify and demonstrate the probability of nonce collisions; explain why nonce uniqueness is critical in CTR.
Task 6	Authenticated Encryption (AES-GCM)	Show that confidentiality alone is insufficient; demonstrate tamper detection with AEAD.
Task 7	Performance Benchmarking	Measure speed/overheads across ECB/CBC/CTR/GCM and analyze security-performance trade-offs.

This lab bridges mathematical foundations, cryptographic security theory, implementation practice, and attack construction. Rather than treating AES and its modes as black-box tools, you will analyze how and why symmetric encryption achieves security — and how subtle misuse can completely undermine it.
By the end of this lab, you should not only be able to implement AES-based encryption schemes, but also rigorously explain the security guarantees they provide and the precise mathematical reasons those guarantees fail when misused.

3 Objectives

The objective of this laboratory is to develop a rigorous understanding of modern symmetric encryption beyond basic implementation. By the end of this lab, you should be able to formally analyze block cipher constructions, experimentally evaluate their security properties, and demonstrate how improper usage leads to concrete and exploitable vulnerabilities.
Specifically, you will:

- Understand the mathematical structure of a block cipher defined as:

  $$E_k : \{0,1\}^n \to \{0,1\}^n$$

  and explain why determinism alone is insufficient for securely encrypting multi-block messages.

- Analyze why Electronic Codebook (ECB) fails semantic security by showing:

  $$P_i = P_j \Rightarrow C_i = C_j$$

  and construct a practical IND-CPA distinguisher to empirically demonstrate structural leakage.

- Evaluate Cipher Block Chaining (CBC) mode and explain the importance of a fresh, unpredictable initialization vector (IV):

  $$C_1 = E_k(P_1 \oplus IV)$$

  and experimentally demonstrate how IV reuse compromises confidentiality and introduces linkability.

- Implement Counter (CTR) mode and explain its stream-like construction:

  $$C_i = P_i \oplus E_k(\text{Nonce} \parallel \text{Counter}_i)$$

  including why encryption and decryption are identical operations.

- Demonstrate how nonce reuse in CTR results in:

  $$C_1 \oplus C_2 = P_1 \oplus P_2$$

  and recover partial plaintext under a two-time pad scenario.

- Analyze nonce collision probability using the birthday bound:

$$\Pr[\text{collision}] \approx \frac{q^2}{2^{r+1}},$$

  and experimentally demonstrate how increasing the number of encryptions increases the likelihood of keystream reuse in CTR mode.

- Distinguish between confidentiality and integrity by comparing CBC with authenticated encryption (AES-GCM), and explain why decryption must reject tampered ciphertexts:

$$\mathrm{Dec}_k(C, T) = \begin{cases} P & \text{if tag valid} \\ \perp & \text{otherwise} \end{cases}$$

- Measure and interpret performance trade-offs across ECB, CBC, CTR, and GCM modes, and relate these observations to practical deployment decisions.

This lab is designed to bridge cryptographic computational security mathematical theory, implementation practice, and adversarial thinking. Rather than treating AES as a black box, you will analyze the precise conditions under which symmetric encryption achieves security and the equally precise conditions under which it fail

4 Lab Requirements

This laboratory requires both correct implementation and rigorous analysis. You are expected to demonstrate technical accuracy, reproducibility, and clear reasoning throughout your submission.

4.1 Technical Environment

- A Linux-based virtual machine (Kali Linux or Ubuntu recommended).
- Python 3.x installed.
- The `cryptography` Python library.
- Standard command-line tools (e.g., `sha256sum`, `xxd`, `time`).

All cryptographic operations must be performed programmatically using Python. External tools (e.g., OpenSSL) may be used only for verification purposes, not as a replacement for implementation.

4.2 Implementation Requirements

- You must implement and test AES in the following modes:
  - ECB
  - CBC
  - CTR
  - GCM (Authenticated Encryption)
- Keys must be generated using secure randomness.
- IVs (for CBC) and nonces (for CTR/GCM) must be handled explicitly. You must demonstrate both:
  - Correct usage (fresh random IV/nonce).
  - Misuse scenarios (IV or nonce reuse).
- For CBC mode, correct padding must be implemented (e.g., PKCS#7).
- For every encryption/decryption experiment, correctness must be verified using cryptographic hashes:

$$H(P_{\text{original}}) = H(P_{\text{recovered}})$$

4.3 Experimental Evidence

Each task must include:

- Terminal screenshots showing execution.
- Relevant ciphertext excerpts (hex format).
- Hash verification outputs.
- Clear indication of IVs/nonces used.
- A short explanation of what the experiment demonstrates.

4.4 Attack Demonstrations

For misuse scenarios (e.g., CTR nonce reuse), you must:

- Clearly state the violated security assumption.
- Show the mathematical relationship exploited (e.g., $C_1 \oplus C_2 = P_1 \oplus P_2$).
- Provide a step-by-step recovery process.
- Recover at least 20 bytes of unknown plaintext (where required).

4.5 Performance Evaluation

You are required to conduct a systematic performance analysis of the implemented encryption modes. The objective of this section is not only to measure execution time, but also to understand how structural differences between block cipher modes affect computational efficiency and scalability.
You must benchmark both **encryption and decryption** for at least three different file sizes (for example: 1KB, 1MB, and 10MB). These file sizes are selected to illustrate:

- Small-input overhead behavior,
- Medium-scale processing performance,
- Large-scale throughput characteristics.

For each file size and for each mode (ECB, CBC, CTR, and GCM), you must report:

- Average execution time (repeat each measurement at least 5 times and compute the average),
- Encryption time versus decryption time comparison,
- Mode comparison across different file sizes,
- Observations regarding parallelism, chaining dependencies, and computational efficiency.

When analyzing performance, consider the following:

- CBC encryption is inherently sequential due to block chaining.
- CTR and GCM allow parallel processing of blocks.
- GCM introduces additional computational overhead due to authentication (GHASH computation).
- ECB may appear fast but is cryptographically insecure.

Your results must be presented in a structured table clearly indicating:

- File size,
- Encryption mode,
- Average encryption time,
- Average decryption time.

After presenting the results, include a brief analytical discussion (1–2 paragraphs) explaining the observed trends and relating them to the internal design properties of each mode (parallelism, chaining structure, and authentication overhead). The purpose of this section is to connect theoretical cryptographic design principles with empirical performance measurements.

5 Lab Tasks

This section describes the tasks that must be completed for this assignment. Each task builds upon the previous one and is designed to demonstrate both correct use and misuse of symmetric encryption schemes. You are expected to complete **all tasks** and provide experimental evidence, explanations, and analysis as specified.

Important: Unless a task explicitly says "use the provided ciphertext package," you are expected to **generate your own plaintext files** and run your own experiments. Misuse/attack tasks may require you to use the **group-specific** artifacts downloaded from Canvas.

5.1 Task 1: ECB Chosen-Plaintext Distinguisher

Objective: Demonstrate why deterministic block cipher encryption (ECB mode) fails to achieve semantic security (IND-CPA).

5.1.1 Background (Why ECB fails)

In ECB mode:

$$C_i = E_k(P_i)$$

Because encryption is deterministic:

$$P_i = P_j \Rightarrow C_i = C_j$$

Therefore, ECB leaks equality/structure across blocks, enabling distinguishers.

5.1.2 Step-by-Step Instructions

Step 1: Create two plaintexts of equal length (multiple of 16 bytes).

Work inside your Kali/Ubuntu VM terminal. Create a working folder:

```
mkdir -p lab_ecb && cd lab_ecb
```

Create P_0 as repeated identical 16-byte blocks (AES block size is 16 bytes):

```
printf 'AAAAAAAAAAAAAAAA'%.0s {1..8} > P0.txt
```

Create P_1 as random-looking bytes of the **same size**:

```
head -c 128 /dev/urandom > P1.txt
```

Verify both are equal length:

```
ls -l P0.txt P1.txt
```

Step 2: Generate an AES key.

Generate a random 16-byte (AES-128) key in hex (you may also use AES-256 if instructed):

```
python3 - << 'PY'
import os, binascii
print(binascii.hexlify(os.urandom(16)).decode())
PY
```

Save the printed hex key into your notes (do not publish secret keys in public places).

Step 3: Encrypt both plaintexts under AES-ECB using the same key.

Using your Python implementation (required) encrypt P_0 and P_1 in ECB mode and save:

- `C0.bin` = AES-ECB(P_0)
- `C1.bin` = AES-ECB(P_1)

Step 4: Block analysis (detect repeated ciphertext blocks).

Split ciphertext into 16-byte blocks and count duplicates. If repeated blocks exist, guess it came from P_0.

Step 5: Build the distinguisher.

Your distinguisher function should:

- parse ciphertext into 16-byte blocks,
- compute duplicate count,
- output guess $b' \in \{0, 1\}$.

Step 6: Run multiple trials and compute success rate.

Repeat at least 20 trials:

1. choose random $b \in \{0, 1\}$,
2. encrypt P_b under ECB,
3. run distinguisher and output guess b',
4. record whether $b' = b$.

Compute:

$$\text{Success Rate} = \frac{\#\text{Correct}}{\#\text{Trials}}$$

and optionally IND-CPA advantage:

$$\text{Adv}_{\mathcal{A}}^{\text{IND-CPA}} = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

What to Submit

- Your ECB encryption code and distinguisher code.
- Screenshots showing ciphertext block repetition for P_0.
- A results table (at least 20 trials) and computed success rate.
- 1–2 paragraphs explaining why ECB is not IND-CPA secure.

5.2 Task 2: CBC Mode with Random vs Reused IV

Objective: Analyze the importance of IV randomness in CBC mode.

5.2.1 Background-Why CBC may be good

CBC encrypts:

$$C_1 = E_k(P_1 \oplus IV), \qquad C_i = E_k(P_i \oplus C_{i-1}) \text{ for } i \geq 2.$$

If the same IV is reused for the same first block P_1, encryptions become linkable.

5.2.2 Step-by-Step Instructions

Step 1: Prepare a plaintext file.

Create a plaintext of at least a few blocks (e.g., 256 bytes):

```
head -c 256 /dev/urandom > msg.bin
```

Step 2: Encrypt twice using CBC with fresh random IVs.

Generate IV_1 and IV_2 randomly (16 bytes each), encrypt the same plaintext twice:

$$C^{(1)} = \mathrm{CBCEnc}_k(msg, IV_1), \quad C^{(2)} = \mathrm{CBCEnc}_k(msg, IV_2).$$

Show that ciphertexts differ (hash them).

Step 3: Encrypt twice using CBC with a fixed IV.

Now reuse the same IV:

$$C^{(3)} = \mathrm{CBCEnc}_k(msg, IV), \quad C^{(4)} = \mathrm{CBCEnc}_k(msg, IV).$$

Show ciphertexts are identical (or at minimum linkable at the beginning).

Step 4: Verification.

Use hashes:

```
sha256sum msg.bin
sha256sum recovered.bin
```

and show successful decryption.

What to Submit

- CBC encryption/decryption code (with PKCS#7 padding).
- Evidence (hashes/screenshots) that random IV gives different ciphertexts.
- Evidence that IV reuse makes outputs linkable.
- Short explanation (why CBC needs fresh IVs).

5.3 Task 3: CTR Mode Implementation and Correctness

Objective: Implement AES-CTR correctly and verify correctness using hashes.

5.3.1 Background

CTR mode uses:

$$C_i = P_i \oplus E_k(\text{Nonce} \,\|\, \text{Counter}_i), \qquad P_i = C_i \oplus E_k(\text{Nonce} \,\|\, \text{Counter}_i).$$

Encryption and decryption are identical (XOR with the same keystream).

5.3.2 Step-by-Step Instructions

Step 1: Create a test file.

```
head -c 4096 /dev/urandom > ctr_msg.bin
```

Step 2: Choose key and nonce.

Generate a random AES key and a nonce (size per your implementation choice; document it).

Step 3: Encrypt and decrypt.

Produce `ctr_ct.bin` and then decrypt to `ctr_recovered.bin`.

Step 4: Verify correctness using hashes.

```
sha256sum ctr_msg.bin ctr_recovered.bin
```

What to Submit

- CTR code + screenshots.
- Hash verification that original equals recovered.
- Short explanation: why CTR behaves like a stream cipher.

5.4 Task 4: CTR Nonce Reuse Attack (Two-Time Pad)

Objective: Demonstrate how nonce reuse in CTR leads to catastrophic leakage.

5.4.1 Background

If the same nonce is reused for two messages, the keystream repeats:

$$C_1 = P_1 \oplus K, \qquad C_2 = P_2 \oplus K \Rightarrow C_1 \oplus C_2 = P_1 \oplus P_2.$$

5.4.2 Step-by-Step Instructions

Step 1: Obtain two ciphertexts produced with the same key and nonce.

Use your **group-provided** files (Ciphertext.zip) or generate them exactly as instructed.

Step 2: XOR the ciphertexts.

Write a Python script to compute:

$$X = C_1 \oplus C_2.$$

Save output as `xor_ct.bin`.

Step 3: Inspect for structure.

Use:

```
xxd xor_ct.bin | head
strings xor_ct.bin | head
```

Step 4: Recover plaintext (at least 20 bytes).

Use a known-plaintext or guessed-plaintext approach (crib dragging):

- Guess a plausible word/segment in P_1 at some offset,
- XOR it with X to derive the corresponding segment in P_2,
- Iterate until you recover at least 20 bytes of unknown plaintext.

What to Submit

- XOR script and results.
- Evidence of recovered plaintext bytes (with offsets).
- Explanation of why nonce reuse recreates a "two-time pad" failure.

5.5 Task 5: Birthday Attack on CTR (Nonce Collision)

Objective: Understand and demonstrate how nonce collisions become likely as the number of encryptions grows (birthday phenomenon), and why CTR requires nonce uniqueness.

5.5.1 Background (Birthday bound)

Suppose CTR uses an r-bit nonce chosen uniformly at random for each encryption. After q encryptions, the probability of **at least one nonce collision** is approximately:

$$\Pr[\text{collision}] \approx 1 - \exp\left(-\frac{q(q-1)}{2 \cdot 2^r}\right) \approx \frac{q^2}{2^{r+1}} \quad \text{(for small probabilities).}$$

A collision means the same nonce repeats, causing keystream reuse and enabling the Task 4 attack.

5.5.2 Step-by-Step Instructions

Step 1: Use a deliberately small nonce size for the experiment.

To observe collisions in a lab setting, you will simulate CTR with a small nonce space (e.g., $r = 16$ bits). **This is for demonstration only** (real systems use much larger nonces).

Step 2: Run repeated encryptions and record nonces.

Write a Python script that:

- generates a random r-bit nonce per encryption,
- stores nonces in a set,
- stops when a collision occurs,
- records the trial count q at collision.

Step 3: Repeat the experiment (multiple runs).

Run at least 20 independent experiments and record the collision points:

$$q_1, q_2, \ldots, q_{20}.$$

Compute the average $\overline{q}$ and compare it to the birthday intuition:

$$q \approx 1.2 \cdot 2^{r/2} \quad \text{(rough collision scale).}$$

Step 4: Connect collision to keystream reuse.

Demonstrate (at least once) that when the nonce repeats under CTR with the same key, you can apply:

$$C_1 \oplus C_2 = P_1 \oplus P_2$$

and recover structure (as in Task 4).

What to Submit

- Your collision-simulation script.
- A table of at least 20 runs listing collision trial counts q_i.
- Your computed $\overline{q}$ and a comparison to the birthday estimate.
- A short explanation connecting nonce collisions to CTR insecurity.

5.6 Task 6: Integrity vs Confidentiality (AES-CBC vs AES-GCM)

Objective: Demonstrate that encryption alone does not provide integrity and show how AES-GCM detects tampering.

5.6.1 Background

Authenticated Encryption produces ciphertext and a tag:

$$(C, T) = \mathrm{Enc}_k(P, A), \quad \mathrm{Dec}_k(C, T, A) = \begin{cases} P & \text{tag valid} \\ \perp & \text{otherwise.} \end{cases}$$

5.6.2 Step-by-Step Instructions

Step 1: CBC tampering demonstration.

Encrypt a message with CBC. Flip 1 byte in ciphertext and decrypt. Observe that CBC decryption may output corrupted plaintext (and sometimes padding failure), but **does not guarantee detection**.

Step 2: GCM tampering demonstration.

Encrypt the same message with GCM. Flip 1 byte in ciphertext (or tag) and attempt decryption. It must fail with $\perp$ (authentication failure).

Step 3: Explain the difference.

Write a short explanation distinguishing confidentiality vs integrity and why AEAD is modern best practice.

What to Submit

- CBC tamper evidence (screenshots + outputs).
- GCM tamper evidence (authentication failure).
- Short comparison table and explanation.

5.7 Task 7: Performance Benchmarking (ECB/CBC/CTR/GCM)

Objective: Measure practical performance and analyze security-performance trade-offs.

5.7.1 Step-by-Step Instructions

Step 1: Prepare datasets.

Create three file sizes (example):

```
head -c 1024 /dev/urandom > data_1KB.bin
head -c 1048576 /dev/urandom > data_1MB.bin
head -c 10485760 /dev/urandom > data_10MB.bin
```

Step 2: Benchmark each mode.

For each file size and each mode (ECB/CBC/CTR/GCM), measure encryption and decryption time (repeat 5 times and average).

Step 3: Report results.

Create a table with:

- file size,
- mode,
- enc time (avg),
- dec time (avg).

Step 4: Analyze.

Write 1–2 paragraphs explaining observed differences (e.g., CTR/GCM parallelism vs CBC chaining).

What to Submit

- Benchmark script or documented commands.
- Results table and analysis discussion.

Submission Requirements

- Submit a single **PDF report** on Canvas by the stated deadline. Only one group member should submit on behalf of the group.
- Your report must be clearly structured and organized by task (Task 1 through Task 7).
- For each task, your report must include:
  - A clear explanation of what was implemented and why (written in complete sentences).
  - Relevant mathematical expressions where appropriate.
  - Screenshots showing execution in the **Kali Linux / Ubuntu terminal**.
  - Evidence of encryption and decryption correctness (where required).
  - SHA-256 hash outputs used for verification.
  - Interpretation of results (not just raw output).
- **Task-Specific Requirements:**
  - **Task 1 (ECB Distinguisher):**

    - Screenshots showing repeated ciphertext blocks.
    - Output of duplicate block counts.
    - IND-CPA success rate and computed advantage.
  - **Task 2 (CBC IV Experiment):**
    - Evidence of ciphertext difference under fresh IVs.
    - Evidence of identical (or linkable) ciphertext under reused IV.
    - Hash verification of successful decryption.
  - **Task 3 (CTR Implementation):**
    - Key and nonce size clearly documented.
    - Encryption and decryption outputs.
    - Matching SHA-256 hashes for original and recovered files.
  - **Task 4 (CTR Nonce Reuse Attack):**
    - XOR output file evidence.
    - Demonstration of plaintext recovery (at least 20 bytes).
    - Clear explanation of why nonce reuse causes keystream reuse.
  - **Task 5 (Birthday Collision Experiment):**
    - At least 20 experimental runs recorded.
    - Average collision point calculation.
    - Comparison with theoretical birthday bound.
  - **Task 6 (Authenticated Encryption - AES-GCM):**
    - Demonstration of successful encryption and decryption.
    - Evidence that tampering results in decryption failure ($\perp$).
  - **Task 7 (Performance Benchmarking):**
    - Timing results for ECB, CBC, CTR, and GCM.
    - Structured table summarizing measurements.
    - Brief analysis of security-performance trade-offs.

- Your Python source code must be included either:
  - zipped, or
  - as a file.
- All results must be reproducible. If your steps cannot be followed or verified, points may be reduced.
- Work that cannot be verified (e.g., missing screenshots, missing code, unclear reasoning, or undocumented parameters) may receive reduced points even if the final outputs appear correct.
- Each group must use only their assigned ciphertext package for Task 4. Outputs will differ across groups.

Grading Rubric (Total: 100 Points)

Task	Assessment Criteria	Points
Task 1: ECB Distinguisher (15 pts)	Correct AES-ECB implementation and encryption of test plaintexts	5
	Correct detection of repeated ciphertext blocks	4
	Proper IND-CPA reasoning and success rate analysis	3
	Clear explanation of why ECB violates semantic security	3
Task 2: CBC IV Experiment (15 pts)	Correct CBC implementation with proper padding (PKCS#7)	5
	Demonstration of fresh IV producing different ciphertexts	3
	Demonstration of IV reuse causing linkability	3
	Clear explanation of IV importance and security impact	4
Task 3: CTR Mode Implementation (10 pts)	Correct AES-CTR implementation	4
	Proper nonce handling and documentation	2
	Correct hash-based verification of encryption/decryption	2
	Explanation of stream-like behavior	2
Task 4: CTR Nonce Reuse Attack (20 pts)	Correct computation of $C_1 \oplus C_2$	5
	Clear demonstration of keystream reuse	4
	Recovery of at least 20 bytes of unknown plaintext	6
	Mathematical explanation of the two-time pad vulnerability	5

Task	Assessment Criteria	Points
Task 5: Birthday Attack on CTR (15 pts)	Correct implementation of nonce-collision simulation	5
	At least 20 experimental runs recorded	3
	Correct computation of average collision point	3
	Comparison with theoretical birthday bound	4
Task 6: Authenticated Encryption (AES-GCM) (10 pts)	Correct GCM implementation	3
	Demonstration of tamper detection failure ($\perp$)	4
	Clear distinction between confidentiality and integrity	3
Task 7: Performance Benchmarking (10 pts)	Benchmark across required file sizes	4
	Proper timing methodology (multiple runs + averages)	3
	Structured results table	2
	Clear analysis of performance-security trade-offs	1
Report Quality (5 pts)	Clear structure and logical organization	2
	Screenshots and reproducible outputs included	2
	Code readable and well formatted	1
Total		**100**